

Module : Data Mining

Filière : Cycle d'Ingénieur - 2A

RAPPORT DE TP 1

Implémentation des Algorithmes

Prétraitement, Statistiques et Mesures de Similarité

Réalisé par :

Youssef FELLAH

Encadré par :

Pr. Hamza ER-RAHMOUNY

Année universitaire : 2025/2026

1 Introduction

Ce rapport présente la résolution complète du premier TP de Data Mining. L'objectif est de manipuler les structures de données Python et de recoder manuellement les fonctions statistiques et de normalisation afin de comprendre les fondements du traitement de données avant l'utilisation de bibliothèques spécialisées.

2 Exercice 1 : Structures de Données

2.1 Code Python

```
# 1. Creation de L1
L1 = [2, 5, 7, 9, 10, 5, 15, 2, 18, 49, 17, 4]

# 2. Construction de L2 (carres)
L2 = [x**2 for x in L1]

# 3. Creation du dictionnaire D
D = {k: v for k, v in zip(L1, L2)}

# 4. Rajout de L2 a L1 (Deux methodes)
# Methode 1: Extension (in-place)
L1_copy1 = list(L1)
L1_copy1.extend(L2)
# Methode 2: Concatenation
L1_copy2 = L1 + L2

# 5. Conversion en ensemble S1
S1 = set(L1_copy1)

# 6. Suppression des elements pairs (Methode sure)
# On cree un nouvel ensemble filtre pour eviter l'erreur de modification
#      durant l'iteration
S1 = {x for x in S1 if x % 2 != 0}

# 7. Nettoyage du dictionnaire D
for k in list(D.keys()):
    if k in S1:
        del D[k]

# 8 & 9. Ajout de la taille n dans D
n = len(S1)
D[n] = "S1"

print(f"Dictionnaire final: {D}")
```

2.2 Réponses aux Questions d'Analyse

Question 3 : Si une clé apparaît plusieurs fois dans L1, c'est la valeur associée à la **dernière occurrence** qui restera dans D. En Python, les dictionnaires ne tolèrent pas de doublons pour les clés; chaque nouvelle affectation sur une clé existante écrase la valeur précédente.

Question 5 : Lors de la transformation en `set`, les doublons sont **supprimés** et l'ordre des éléments est **perdu**. Un ensemble est une collection non ordonnée d'éléments uniques.

Question 6 (Méthode sûre) : Pour ne pas modifier `S1` pendant l'itération, la méthode la plus propre consiste à utiliser une *compréhension d'ensemble* pour créer un nouveau set filtré, ou à itérer sur une copie (`list(S1)`).

3 Exercice 2 : Statistiques Descriptives

Implémentation native des formules sans utiliser `sum()` ou `sorted()`.

3.1 Code Source

```
x = [5.1, 4.9, 6.2, 5.9, 6.7, 5.0, 6.4, 7.0, 5.4, 6.9]

def tri_bulles(liste):
    res = list(liste)
    n = len(res)
    for i in range(n):
        for j in range(0, n - i - 1):
            if res[j] > res[j + 1]:
                res[j], res[j + 1] = res[j + 1], res[j]
    return res

def somme(liste):
    total = 0.0
    for val in liste:
        total += val
    return total

def moyenne(liste):
    return somme(liste) / len(liste)

def median(liste):
    triee = tri_bulles(liste)
    n = len(triee)
    mid = n // 2
    return triee[mid] if n % 2 != 0 else (triee[mid-1] + triee[mid]) / 2

def var_pop(liste):
    mu = moyenne(liste)
    return somme([(val - mu)**2 for val in liste]) / len(liste)

def var_sample(liste):
    mu = moyenne(liste)
    return somme([(val - mu)**2 for val in liste]) / (len(liste) - 1)

def std_pop(liste): return var_pop(liste)**0.5
def std_sample(liste): return var_sample(liste)**0.5
```

3.2 Résultats Attendus

- Moyenne : 5.95
- Médiane : 6.05

— Écart-type Population (σ) : 0.7658

4 Exercice 3 : Normalisation des Données

4.1 Code d'Exécution et Affichage

```
def minmax_scale(x):
    mi, ma = min(x), max(x)
    return [0.0]*len(x) if mi == ma else [(v - mi)/(ma - mi) for v in x]

def zscore_pop(x):
    mu, sig = moyenne(x), std_pop(x)
    return [0.0]*len(x) if sig == 0 else [(v - mu)/sig for v in x]

def robust_iqr(x):
    triee = tri_bulles(x)
    q2 = median(triee)
    q1 = median(triee[:len(triee)//2])
    q3 = median(triee[(len(triee)+1)//2:] if len(triee)%2!=0 else triee[len(triee)//2:])
    return [0.0]*len(x) if q3 == q1 else [(v - q2)/(q3 - q1) for v in x]

# Affichages requis
x_norm = minmax_scale(x)
print(f"Min-Max: min={min(x_norm)}, max={max(x_norm)}")
x_z = zscore_pop(x)
print(f"Z-Score: moyenne={moyenne(x_z):.2f}, std={std_pop(x_z):.2f}")
x_r = robust_iqr(x)
print(f"Robust: mediane={median(x_r)}")
```

5 Exercice 4 : Distances et Similarités

5.1 Implémentation

```
def dist_euclid(x, y):
    return sum([(a - b)**2 for a, b in zip(x, y)])**0.5

def dist_manhattan(x, y):
    return sum([abs(a - b) for a, b in zip(x, y)])

def sim_jaccard_binary(a, b):
    inter = sum([1 for i, j in zip(a, b) if i == 1 and j == 1])
    union = sum([1 for i, j in zip(a, b) if i == 1 or j == 1])
    return inter / union if union != 0 else 0.0

# Matrice D (5x5)
x1 = [5.1, 3.5, 1.4, 0.2]
x2 = [4.9, 3.0, 1.4, 0.2]
x3 = [6.2, 2.9, 4.3, 1.3]
x4 = [5.9, 3.0, 4.2, 1.5]
x5 = [6.7, 3.1, 5.6, 2.4]

vects = [x1, x2, x3, x4, x5]
matrice_D = [[dist_euclid(vi, vj) for vj in vects] for vi in vects]
```

5.2 Comparaison x1 et x2

- **Manhattan (x1, x2) : 0.7**
- **Euclidienne (x1, x2) : 0.5385**
- **Analyse :** La distance de Manhattan est supérieure à l'Euclidienne car elle calcule la somme des écarts absolus (trajet "en grille") tandis que l'Euclidienne calcule le trajet direct en ligne droite.

6 Exercice 5 : Corrélation et Régression

6.1 Calcul de Pearson

```
y = [1.4, 1.4, 4.3, 4.2, 5.6, 1.3, 4.5, 5.4, 1.7, 5.1]

def pearson_corr(x, y):
    mu_x, mu_y = moyenne(x), moyenne(y)
    num = sum([(xi - mu_x) * (yi - mu_y) for xi, yi in zip(x, y)])
    den_x = sum([(xi - mu_x)**2 for xi in x])**0.5
    den_y = sum([(yi - mu_y)**2 for yi in y])**0.5
    return num / (den_x * den_y)
```

Le coefficient de Pearson calculé est $r \approx 0.9565$. **Explication (Q2.c) :** Cette valeur proche de 1 indique une **très forte corrélation linéaire positive** entre x et y .

6.2 Régression Linéaire

```
n = len(x)
sum_x, sum_y = somme(x), somme(y)
sum_xy = sum([xi * yi for xi, yi in zip(x, y)])
sum_x2 = sum([xi**2 for xi in x])

a = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x**2)
b = moyenne(y) - a * moyenne(x)
```

- **Pente a : 2.1782**
- **Intercept b : -9.4701**

Relation (Q3.b) : La relation est positive; quand x augmente d'une unité, y augmente en moyenne de 2.1782 unités.

Prédiction pour $x_0 = 6.0$: $y = (2.1782 \times 6.0) - 9.4701 = 3.5991$.

7 Conclusion

Ce TP a permis de valider la maîtrise des outils de base du Data Mining en environnement Python. L'implémentation manuelle des formules mathématiques assure une base solide pour l'utilisation future de modèles prédictifs plus complexes.